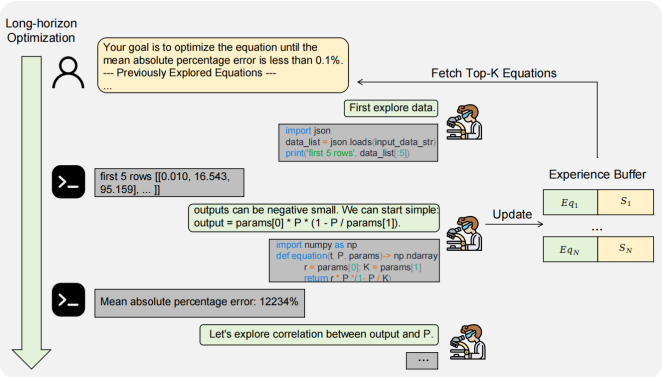
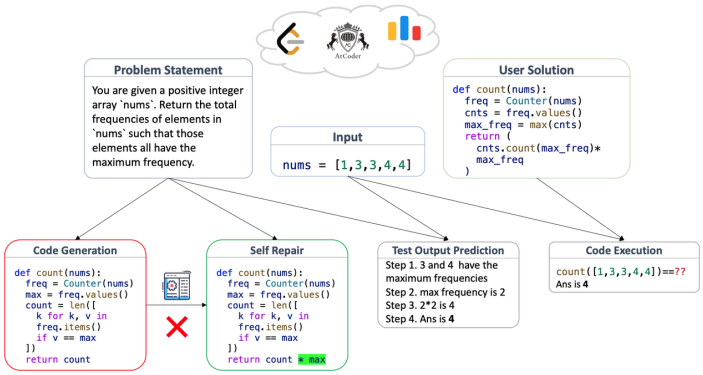
# 提示词工程考试说明（2026年春）

## 背景




一个利用LLM Agent完成真实场景科学发现任务的示例，Agent通过与环境（代码解释器）进行多轮交互，调用多种工具，如数据分析、结果评估，从而发现科学方程式。




一个最简化的LLM与环境交互的场景：代码执行器。对于一道具有难度的代码问题，LLM生成初步的解法，交给代码解释器进行执行判分，并获得反馈；如遇到错误反馈信息再进行下一轮次的修正。

在人工智能快速发展的浪潮中，大语言模型（LLM）已成为推动技术变革的核心力量。LLM 不仅在自然语言处理任务上取得了 SOTA 表现，其强大的上下文学习（In-context Learning）和零/少样本（Zero/Few-shot）学习能力也为解决复杂的编程与推理问题开辟了新的途径。与此同时，**AI Agent（智能体）** 正在成为 LLM 应用的重要范式。与传统的单轮问答不同，Agent 能够自主地进行多步推理、调用外部工具（如代码执行器、搜索引擎等）、根据环境反馈动态调整策略，从而解决更加复杂的现实任务。这种"感知—思考—行动—反馈"的闭环能力，使得 Agent 在代码生成、自动化调试、科学研究等领域展现出巨大潜力。如上左图所示，LLM Agent通过与环境（代码解释器）进行多轮交互，完成符号回归任务，从而更高效地寻找科学方程式。**提示词工程（Prompt Engineering）** 作为驾驭 LLM 和 Agent 能力的关键技术，已从简单的问答交互演变为一门涉及指令设计、示例组织、思维链（Chain-of-Thought）引导、工具调用编排和多轮策略设计的专业学科。

## 考试内容

### 数据集介绍

本次考核考虑一个简易的大模型与环境交互的场景——利用提示词工程使大语言模型解决 **代码编程题目（如上右图所示）**。每道题目要求模型生成一段 Python 代码，使其能够通过所有隐藏的测试用例。考生将获得一个验证集文件（附件：`data/dev.jsonl`），该文件中每行代表一个独立的代码任务。每一条数据的字段说明：

代码块

```
id: 题目的编号
prompt: OpenAI Message格式的列表
tests: 输入输出测试用例字典
```

## 客观评价指标

评价指标为 **pass@1**，即模型生成的代码能否通过所有测试用例：

- 通过全部测试用例：得分 1
- 未通过：得分 0

考生的总分为测试集上所有题目 pass@1 的平均值。为保证结果稳定性，每道题会进行多次采样取平均（默认取16次）

## 考核方式

考生需要编辑 **`solution.py`**（这是你唯一需要修改的文件），实现其中的 `run_question` 函数。

该函数接收以下参数：

| 参数 | 类型 | 说明 |
| --- | --- | --- |
| `question_prompt` | `str` | 题目描述文本 |
| `call_llm` | `callable` | 调用大语言模型，输入 messages 列表，返回回复文本 |
| `execute_code` | `callable` | 代码执行器，输入包含代码块的文本，返回测试结果 |
| `max_turns` | `int` | 最大允许的 LLM 调用轮数 |
| `count_tokens` | `callable` | Token 计数器，输入文本，返回 token 数量 |

函数需要返回一个包含 ````python ... ```` 代码块的字符串作为最终答案。

## 可用工具详细说明

### 1. `call_llm(messages: list[dict]) -> str`

调用大语言模型。输入 OpenAI 格式的 messages 列表，返回模型回复文本。

代码块

```
response = call_llm([
```

```
    {"role": "user", "content": "请解决以下编程题..."}
])
```

注意：模型的 temperature、top_p、max_tokens 等参数已固定，考生只能通过 messages 内容影响模型输出。LLM 调用次数受 `max_turns` 硬性限制**（最多3次）**，超出后调用将失败。

**2. `execute_code(response: str) -> dict`**

代码执行器。输入包含 ```python ... ``` 代码块的文本，在沙箱中提取并运行代码，返回测试结果。

代码块

```
result = execute_code(response)
# result["passed"]   -> bool, 是否通过所有测试用例
# result["feedback"] -> str,  未通过时的错误信息/测试反馈
```

**3. `count_tokens(text: str) -> int`**

计算文本的 token 数量，可用于管理上下文长度。注意：单次调用LLM时，输入的prompt长度不得超过16k（以Qwen3-8B tokenizer计数）

代码块

```
n = count_tokens("Hello, world!")
```

### 评分入口

最终评分，会统一运行 `run.py` （除了 `solution.py` 的其余所有代码文件会统一使用原始代码），考生在开发过程中，可以接入自己的LLM API，通过 `run.py` 进行测试迭代。

**PS: 整个运行代码，均采用OpenAI API的风格进行代码的调用。考试过程，LLM API需要考生自备（可以用公开平台的API服务，也可以自行用sglang/vllm进行部署）。为最小化考生成本，评分所用的模型为Qwen3-8B (Instruct)，且不开思考模式。**

## 提交要求

考生需要提交：

1. **Python文件：** （模版文件：solution.py）

- 文件中有且仅能包含所需要的run_question函数

- 不允许import第三方库，仅可import Python标准库（如random, re, json等）
- 禁止出现直接编码公开的相应测试集，并采用穷举法搜索官方正确答案（私有测试集也无法在开源数据中找到）；每个代码会经过内容复核，一经发现，该项考核按0分计算

2. **探索报告**：pdf文件，简易记录探索过程，包含不同提示词策略的尝试、效果和分析，作为主观分数的参考之一

**提交方式：**（截止时间 北京时间 3月4号 12:00 中午 ，期间可以多次提交，会自动覆盖先前的提交文件）

1. 进入链接：https://send2me.cn/UY5jfIK7/RdioWYP6ZNDewA
2. 精确填写个人信息，包括报名号、姓名（由填写错误导致的得分缺失，后果自负！）

# 评分标准

本项考核的分数由两部分组成：

1. **客观得分**（占总分80%）
   - 所有考生在私有测试集(test.jsonl，格式与dev.jsonl完全相同，但是考生无法直接读取其中的答案/测试用例内容) 上的平均准确率性能进行排名并赋分
   - 测试统一使用Qwen3-8B (Instruct) **非思考模式**模型，temperature=1.0，top_p=1，max_output_tokens=8192，测试跑分的代码可以详见给定的 `run.py`
   - 每个样本会进行多次采样（默认16次）取平均指标以保证结果稳定性
2. **提示词主观评价得分**（占总分20%）
   - 由专家老师基于指定评价准则进行评分
   - 评价内容包括提示词的创新性、合理性、可解释性等